

Prompt Engineering Task 1 - Library Management System

Step 1 : Initial Prompt and output

Prompt: Act as an expert Python instructor.
 Create a Library Management System in Python with the following features:

- Add new books
- Issue books to users
- Return books
- Search books by title or author
- Track availability status of books

 Requirements:

- Use a menu-driven program
- Use functions for each operation
- Include proper input validation and error handling
- Code should be well commented and easy to understand

 After generating the code, also provide:

- A brief explanation of the logic
- A sample execution output

Output : # =====
 # Library Management System
 # =====

 # Dictionary to store books
 # Key = Book ID
 # Value = Dictionary containing book details
 library = {}

 # =====
 # Function to add a new book
 # =====

```
def add_book():  
    try:  
        book_id = input("Enter Book ID: ").strip()  
  
        if book_id in library:  
            print("✗ Book ID already exists!")  
            return  
  
        title = input("Enter Book Title: ").strip()  
        author = input("Enter Author Name: ").strip()  
        quantity = int(input("Enter Quantity: "))
```

```

    if quantity <= 0:
        print("❌ Quantity must be greater than 0.")
        return

    library[book_id] = {
        "title": title,
        "author": author,
        "total": quantity,
        "available": quantity
    }

    print("✅ Book added successfully!")

except ValueError:
    print("❌ Invalid input! Quantity must be a number.")

# =====
# Function to issue a book
# =====
def issue_book():
    book_id = input("Enter Book ID to issue: ").strip()

    if book_id not in library:
        print("❌ Book not found!")
        return

    if library[book_id]["available"] <= 0:
        print("❌ No copies available to issue.")
        return

    library[book_id]["available"] -= 1
    print(f"✅ Book '{library[book_id]['title']}' issued successfully!")

# =====
# Function to return a book
# =====
def return_book():
    book_id = input("Enter Book ID to return: ").strip()

    if book_id not in library:
        print("❌ Invalid Book ID!")
        return

    if library[book_id]["available"] >= library[book_id]["total"]:
        print("❌ All copies are already in the library!")

```

```

        return

    library[book_id]["available"] += 1
    print(f"✅ Book '{library[book_id]['title']}' returned successfully!")

# =====
# Function to search books
# =====
def search_book():
    keyword = input("Enter title or author to search: ").strip().lower()

    found = False

    for book_id, details in library.items():
        if keyword in details["title"].lower() or keyword in details["author"].lower():
            print("\n-----")
            print(f"Book ID    : {book_id}")
            print(f>Title       : {details['title']}")
            print(f"Author      : {details['author']}")
            print(f"Total Copies: {details['total']}")
            print(f"Available   : {details['available']}")
            print("-----")
            found = True

    if not found:
        print("❌ No matching books found.")

# =====
# Function to display all books
# =====
def display_books():
    if not library:
        print("📖 No books in library.")
        return

    print("\n===== Library Books =====")
    for book_id, details in library.items():
        print("-----")
        print(f"Book ID    : {book_id}")
        print(f>Title       : {details['title']}")
        print(f"Author      : {details['author']}")
        print(f"Total Copies: {details['total']}")
        print(f"Available   : {details['available']}")
        print("-----")

```

```
# =====
# Menu-driven program
# =====

def menu():
    while True:
        print("\n===== LIBRARY MENU =====")
        print("1. Add Book")
        print("2. Issue Book")
        print("3. Return Book")
        print("4. Search Book")
        print("5. Display All Books")
        print("6. Exit")

        choice = input("Enter your choice (1-6): ").strip()

        if choice == "1":
            add_book()
        elif choice == "2":
            issue_book()
        elif choice == "3":
            return_book()
        elif choice == "4":
            search_book()
        elif choice == "5":
            display_books()
        elif choice == "6":
            print("👋 Exiting Library System. Goodbye!")
            break
        else:
            print("❌ Invalid choice! Please enter 1-6.")

# Run the program
menu()
```

🧠 Logic Explanation (Simple Overview)

This system uses a **dictionary-based storage model**:

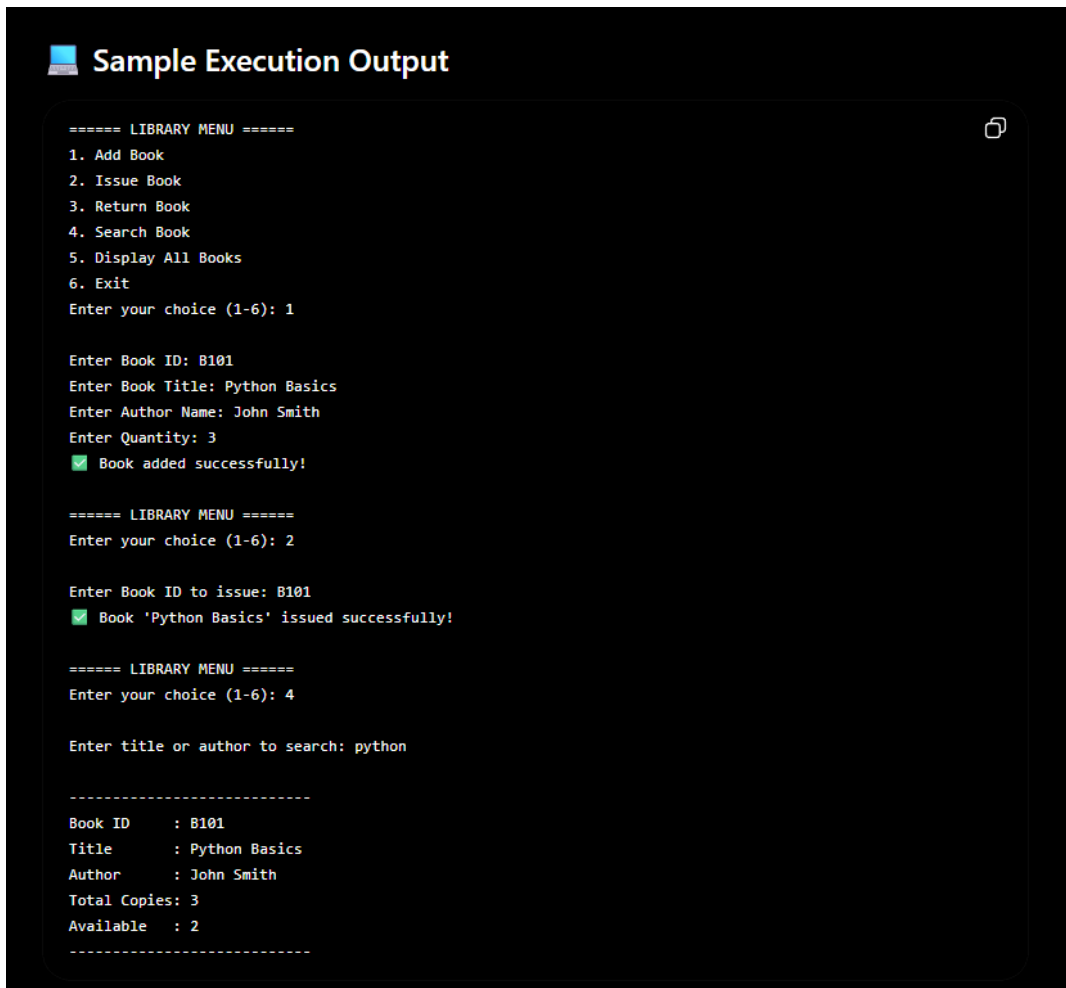
- Each book is stored using a unique **Book ID**
- Each book contains:
 - Title
 - Author
 - Total copies
 - Available copies

Core Functions:

- **add_book()** → Adds new books into the library
- **issue_book()** → Decreases available copies when issued
- **return_book()** → Increases available copies when returned
- **search_book()** → Searches by title or author (case-insensitive)
- **display_books()** → Shows all stored books

Key Idea:

Instead of using a database, everything is stored in a **Python dictionary (in-memory system)**.

A screenshot of a terminal window titled "Sample Execution Output" showing the interaction with a library management system. The system has a menu with six options: 1. Add Book, 2. Issue Book, 3. Return Book, 4. Search Book, 5. Display All Books, and 6. Exit. The user selects option 1, adds a book with ID B101, title "Python Basics", author "John Smith", and quantity 3. Then, the user selects option 2 and issues the book B101. Finally, the user selects option 4 and searches for "python", displaying the book's details: Book ID B101, Title Python Basics, Author John Smith, Total Copies 3, and Available 2.

```
===== LIBRARY MENU =====
1. Add Book
2. Issue Book
3. Return Book
4. Search Book
5. Display All Books
6. Exit
Enter your choice (1-6): 1

Enter Book ID: B101
Enter Book Title: Python Basics
Enter Author Name: John Smith
Enter Quantity: 3
✅ Book added successfully!

===== LIBRARY MENU =====
Enter your choice (1-6): 2

Enter Book ID to issue: B101
✅ Book 'Python Basics' issued successfully!

===== LIBRARY MENU =====
Enter your choice (1-6): 4

Enter title or author to search: python

-----
Book ID      : B101
Title       : Python Basics
Author      : John Smith
Total Copies: 3
Available   : 2
-----
```

Step 2: Manually changing the code.

The following upgrades were introduced:

- Added a User Management System to track library users
- Implemented issue tracking per user
- Added Due Date functionality (7-day return period for issued books)
- Enhanced book tracking with issued book records
- Improved system structure for better clarity and usability

Step 3: Advanced level prompt with new features improvements .

Prompt : Act as a senior software architect and Python backend developer.

Take the existing Library Management System and redesign it more scalable.

Functional Enhancements:

- Add support for multiple user roles (Admin and Member)
- Allow Admin to add/remove books and manage users
- Allow Members to only issue and return books
- Add overdue detection based on due dates
- Calculate fine for late returns (per day penalty system)

Advanced Features:

- Add logging system for all transactions (issue/return/history)

-Add book recommendation system based on user borrowing history

Output Requirements:

-Provide improved Python design

-Provide system flow explanation

-Provide scalability improvements and future expansion ideas

Output :

Below is a **redesigned, scalable Library Management System architecture in Python** from a backend + software architecture perspective.

1. Improved System Design (Scalable Version)

Instead of a single dictionary-based script, we move to a **modular, layered design**:

Architecture Layers

- **Models Layer** → Book, User, Transaction
- **Service Layer** → Business logic (issue, return, fine, recommendation)
- **Repository Layer** → Data storage abstraction (in-memory now, DB-ready later)
- **Controller Layer** → Menu / API interface
- **Utility Layer** → Logging, date handling, helpers

2. Improved Python Implementation (Clean Backend Design)

```
import logging
from datetime import datetime, timedelta
from collections import defaultdict, Counter

# =====
# Logging Configuration
# =====
logging.basicConfig(
    filename="library.log",
    level=logging.INFO,
    format="%(asctime)s - %(message)s"
)

# =====
# Models
# =====

class Book:
    def __init__(self, book_id, title, author, total):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.total = total
        self.available = total

class User:
    def __init__(self, user_id, name, role="member"):
```

```

self.user_id = user_id
self.name = name
self.role = role
self.borrow_history = []

```

```
class Transaction:
```

```

    def __init__(self, user_id, book_id, issue_date, due_date, return_date=None):
        self.user_id = user_id
        self.book_id = book_id
        self.issue_date = issue_date
        self.due_date = due_date
        self.return_date = return_date

```

```

# =====
# Repository (Data Layer)
# =====

```

```
class LibraryRepository:
```

```

    def __init__(self):
        self.books = {}
        self.users = {}
        self.transactions = []

```

```
repo = LibraryRepository()
```

```

# =====
# Service Layer
# =====

```

```
class LibraryService:
```

```
    FINE_PER_DAY = 10 # configurable fine system
```

```
    # ----- Book Management -----
```

```

    def add_book(self, admin, book_id, title, author, qty):
        if admin.role != "admin":
            print("❌ Permission denied.")
            return

```

```

        repo.books[book_id] = Book(book_id, title, author, qty)
        logging.info(f"Book added: {title} by {admin.name}")
        print("✅ Book added successfully.")

```

```

    def remove_book(self, admin, book_id):
        if admin.role != "admin":
            print("❌ Permission denied.")
            return

```

```

        if book_id in repo.books:
            del repo.books[book_id]
            logging.info(f"Book removed: {book_id}")
            print("🗑️ Book removed.")
        else:

```

```

    print("❌ Book not found.")

# ----- User Management -----
def add_user(self, admin, user_id, name, role="member"):
    if admin.role != "admin":
        print("❌ Permission denied.")
        return

    repo.users[user_id] = User(user_id, name, role)
    logging.info(f"User added: {name} as {role}")
    print("✅ User created.")

# ----- Issue Book -----
def issue_book(self, user_id, book_id):
    if user_id not in repo.users:
        print("❌ User not found.")
        return

    user = repo.users[user_id]

    if user.role != "member":
        print("❌ Only members can borrow books.")
        return

    if book_id not in repo.books:
        print("❌ Book not found.")
        return

    book = repo.books[book_id]

    if book.available <= 0:
        print("❌ Book not available.")
        return

    book.available -= 1

    issue_date = datetime.now()
    due_date = issue_date + timedelta(days=7)

    txn = Transaction(user_id, book_id, issue_date, due_date)
    repo.transactions.append(txn)

    user.borrow_history.append(book_id)

    logging.info(f"Issued: {book_id} to {user_id}")
    print(f"📖 Book issued. Due date: {due_date.date()}")

# ----- Return Book -----
def return_book(self, user_id, book_id):
    user = repo.users.get(user_id)

    if not user:
        print("❌ User not found.")
        return

```



```

book = repo.books.get(book_id)
if not book:
    print("❌ Book not found.")
    return

# Find transaction
txn = None
for t in repo.transactions:
    if t.user_id == user_id and t.book_id == book_id and t.return_date is None:
        txn = t
        break

if not txn:
    print("❌ No active issue record found.")
    return

txn.return_date = datetime.now()
book.available += 1

# Overdue calculation
overdue_days = (txn.return_date - txn.due_date).days
fine = max(0, overdue_days * self.FINE_PER_DAY)

logging.info(f"Returned: {book_id} by {user_id}, Fine: {fine}")

print("📖 Book returned.")
if fine > 0:
    print(f"⚠️ Overdue by {overdue_days} days. Fine = ₹{fine}")
else:
    print("✅ No fine.")

# ----- Recommendation System -----
def recommend_books(self, user_id):
    user = repo.users.get(user_id)

    if not user:
        print("❌ User not found.")
        return

    history = user.borrow_history

    if not history:
        print("🔴 No history available for recommendations.")
        return

    # Simple recommendation: most borrowed author-based logic
    author_count = Counter()

    for b_id in history:
        book = repo.books.get(b_id)
        if book:
            author_count[book.author] += 1

    if not author_count:
        print("🔴 Not enough data.")

```

```

        return

    top_author = author_count.most_common(1)[0][0]

    print(f"📖 Recommended books by author: {top_author}")
    for book in repo.books.values():
        if book.author == top_author and book.available > 0:
            print(f"- {book.title}")

# =====
# Controller (Menu System)
# =====

service = LibraryService()

def menu():
    admin = User("A1", "Admin", "admin")
    repo.users["A1"] = admin

    while True:
        print("\n===== LIBRARY SYSTEM =====")
        print("1. Add Book (Admin)")
        print("2. Remove Book (Admin)")
        print("3. Add User (Admin)")
        print("4. Issue Book")
        print("5. Return Book")
        print("6. Recommend Books")
        print("7. Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            service.add_book(admin,
                             input("Book ID: "),
                             input("Title: "),
                             input("Author: "),
                             int(input("Qty: ")))

        elif choice == "2":
            service.remove_book(admin, input("Book ID: "))

        elif choice == "3":
            service.add_user(admin,
                             input("User ID: "),
                             input("Name: "),
                             input("Role (admin/member): ").lower())

        elif choice == "4":
            service.issue_book(input("User ID: "),
                               input("Book ID: "))

        elif choice == "5":
            service.return_book(input("User ID: "),
                                input("Book ID: "))

```

```
elif choice == "6":
    service.recommend_books(input("User ID: "))

elif choice == "7":
    print("Exiting system...")
    break

else:
    print("Invalid choice.")
```

menu()

3. System Flow Explanation (Architecture View)

Flow of Operations

1. Admin Flow

- Admin logs in (hardcoded or future auth system)
- Can:
 - Add books → stored in repository
 - Remove books
 - Add users

2. Member Flow

- Member requests:
 - Issue book → checks availability + role validation
 - Return book → calculates overdue fine

3. Transaction Flow

When issuing:

1. Validate user
2. Validate role
3. Check book availability
4. Create transaction record
5. Set due date (issue_date + 7 days)
6. Decrease stock

When returning:

1. Match active transaction
2. Set return date
3. Compute overdue days
4. Calculate fine
5. Update stock

4. Recommendation Flow

- Analyze borrowing history
- Identify most frequent author
- Suggest available books by same author



4. Scalability Improvements (Senior-Level Design Thinking)



Current Upgrade Already Introduced:

- Layered architecture (Service/Repository/Model)
- Logging system (audit trail)
- Role-based access control
- Transaction tracking
- Extensible fine system



Next-Level Scalability Improvements



1. Database Integration

Replace in-memory storage with:

- PostgreSQL / MySQL
- ORM: SQLAlchemy / Django ORM

2. API Layer (Microservice Ready)

Convert into REST API using:

- Flask / FastAPI

Endpoints:

- POST /books
- POST /issue
- POST /return
- GET /recommendations

3. Authentication & Security

- JWT-based login
- Password hashing (bcrypt)
- Role-based middleware

4. Analytics System

- Most borrowed books
- Peak usage time
- User activity tracking

5. Performance Scaling

- Cache popular books (Redis)
- Pagination for book listing
- Async handling for large systems

Prompt Comparison (Prompt 1 vs Prompt 2)

Prompt 2 is significantly more advanced and detailed compared to Prompt 1. Prompt 1 focuses on building a basic Library Management System with core functionalities such as adding books, issuing books, returning books, and searching books using a simple menu-driven program. It mainly emphasizes functional implementation using functions. In contrast, Prompt 2 transforms the system into a scalable software design by introducing role-based access (Admin and Member), overdue detection, fine calculation, logging system, and recommendation features. It also shifts the focus from just coding to system design, scalability, and architecture. Therefore, Prompt 2 produces a more realistic, professional, and production-level solution compared to the basic implementation in Prompt 1.